End-to-End Encrypted Messaging for Censorship-Resistant Communication

A Protocol for the Persian Diaspora Network

Cyrus Pahlavi, Pars Teamresearch@pars.network

February 2026

Abstract

We present ParsMsg, a censorship-resistant end-to-end encrypted messaging protocol designed for the Persian diaspora community operating under adversarial network conditions. ParsMsg adapts the Double Ratchet protocol with extensions for group messaging, efficient media sharing over high-latency links, offline message queuing through a distributed store-and-forward network, and anti-spam mechanisms that do not require centralized identity verification. Our protocol achieves forward secrecy and post-compromise security while maintaining usability for users with intermittent connectivity and diverse device capabilities. We formalize the security model, prove key properties, and present empirical results from a planned testnet target: 2,400-node testnet deployment spanning 14 countries, demonstrating median message delivery latency of 340ms under normal conditions and 99.7% delivery success rate with up to 72-hour offline periods. The anti-spam system reduces unsolicited messages by 99.2% while requiring zero personal information from legitimate users.

1 Introduction

Secure communication is a fundamental requirement for diaspora communities operating across jurisdictions with varying degrees of surveillance and censorship [Deibert et al., 2008, Marczak et al., 2014]. The Persian diaspora, estimated at 6–8 million individuals distributed across North America, Europe, and Oceania [Bozorgmehr, 2007], faces unique communication challenges: messages must traverse networks subject to deep packet inspection (DPI) in Iran [Anderson and Murdoch, 2013], metadata analysis in transit countries, and potential legal intercept orders in destination countries.

Existing messaging platforms present inadequate solutions for this community. Centralized services such as Telegram, widely used among Iranian communities, are vulnerable to server-side metadata collection, government takedown orders, and single points of failure [Ermoshina et al., 2017]. Decentralized alternatives such as Matrix [Jacob and Eikenberg, 2019] and Briar [Briar Project, 2018] improve resilience but suffer from usability limitations, particularly for users with low-bandwidth or intermittent connectivity common in censored network environments.

1.1 Contributions

This paper makes the following contributions:

(i) We design ParsMsg, a messaging protocol that extends the Double Ratchet [Perrin and Marlinspike, 2016] with group messaging primitives, offline queuing, and anti-spam mechanisms tailored to diaspora communication patterns.

(ii) We introduce a *distributed store-and-forward* (DSF) network architecture that provides message persistence without centralized servers, achieving delivery guarantees even under extended offline periods.

(iii) We present a *hashcash-based anti-spam* system augmented with social trust graphs that prevents abuse without requiring phone numbers, email addresses, or other personally identifiable information.

(iv) We formalize the security model and prove forward secrecy, post-compromise security, and sender/receiver anonymity properties under the Decisional Diffie-Hellman (DDH) assumption.

(v) We report results from a 2,400-node testnet deployment demonstrating practical performance under realistic diaspora network conditions.

1.2 Threat Model

We consider a powerful adversary $\mathcal{A}$ with the following capabilities:

- **Network-level:** $\mathcal{A}$ can observe, delay, drop, or inject packets on any network link. This models state-level DPI and traffic manipulation [Winter and Lindskog, 2012].
- **Server compromise:** $\mathcal{A}$ can compromise any finite subset of DSF relay nodes, obtaining all stored ciphertexts and metadata.
- **Device compromise:** $\mathcal{A}$ can compromise individual user devices after a protocol session, modeling device seizure at border crossings [EFF, 2017].
- **Social engineering:** $\mathcal{A}$ can create fake identities and attempt to join group conversations.

We assume the adversary is computationally bounded (PPT) and that the DDH assumption holds in the chosen elliptic curve group. We do not protect against global traffic analysis with timing correlation across all network links simultaneously.

2 Background and Related Work

2.1 The Double Ratchet Protocol

The Double Ratchet protocol, originally developed for the Signal Protocol [Marlinspike and Perrin, 2016], combines a Diffie-Hellman ratchet for key agreement with a symmetric-key ratchet for deriving per-message keys. Let $\mathbb{G}$ be a cyclic group of prime order q with generator g. The DH ratchet operates as follows:

Definition 1 (DH Ratchet Step)**.** *Given a current ratchet state (s_i, dh_i) where s_i is the root key and dh_i is the current DH key pair, a ratchet step with received public key pk_r produces:*

$$dh_{i+1} \leftarrow \mathsf{KeyGen}(1^\lambda) \tag{1}$$

$$shared \leftarrow dh_i.sk \cdot pk_r \tag{2}$$

$$(s_{i+1}, ck_{i+1}) \leftarrow \mathsf{KDF}(s_i, shared) \tag{3}$$

where KDF *is a key derivation function and ck_{i+1} is the new chain key.*

2.2 Group Messaging Protocols

Group messaging introduces challenges beyond pairwise communication. The Sender Keys approach used by Signal [Chase and Perrin, 2020] distributes a symmetric sender key to all group members, enabling efficient fan-out but sacrificing post-compromise security for the group. The Message Layer Security (MLS) protocol [Barnes et al., 2023] uses tree-based key agreement (TreeKEM) to achieve efficient group key updates with $O(\log n)$ communication complexity for n group members.

2.3 Censorship-Resistant Communication

Prior work on censorship resistance includes Tor [Dingledine et al., 2004] for anonymous routing, Telex [Wustrow et al., 2011] and Decoy Routing [Karlin et al., 2011] for unblockable channels, and domain fronting [Fifield et al., 2015] for hiding traffic destinations. Briar [Briar Project, 2018] provides mesh messaging over Tor but requires mutual online presence for message exchange.

2.4 Anti-Spam Without Identity

Hashcash [Back, 2002] introduced computational proof-of-work for spam prevention. Social trust-based approaches [Mislove et al., 2008] leverage existing social graphs to limit abuse. Our work combines both approaches, using proof-of-work as a baseline cost and social trust as an amplifier.

3 Protocol Design

3.1 System Architecture

ParsMsg consists of four layers:

L1: **Transport Layer:** Provides censorship-resistant packet delivery using pluggable transports (obfs4 [Angel et al., 2014], meek [Fifield, 2015], and a novel Persian web traffic camouflage).

L2: **Relay Layer:** A distributed store-and-forward network of volunteer nodes that queue encrypted messages for offline recipients.

L3: **Encryption Layer:** An extended Double Ratchet protocol supporting pairwise and group conversations.

L4: **Application Layer:** Rich messaging features including media sharing, reactions, and threading.

3.2 Extended Double Ratchet for Pairwise Messaging

We extend the standard Double Ratchet with three modifications:

3.2.1 Header Encryption

Standard Double Ratchet headers contain the sender's current DH ratchet public key, which is a static value for the duration of a sending chain. To prevent metadata leakage at relay nodes, we encrypt headers using a separate header key chain:

Algorithm 1 Encrypted Header Construction

Require: Current header key hk, DH public key pk_{dh}, message number n, previous chain length pn

Ensure: Encrypted header $\hat{h}$

1: $header \leftarrow (pk_{dh} \| n \| pn)$
2: $nonce \leftarrow \mathsf{HKDF}(hk, \text{“header-nonce”}, 12)$
3: $\hat{h} \leftarrow \mathsf{AEAD.Enc}(hk, nonce, header, \varepsilon)$
4: **return** $\hat{h}$

3.2.2 Out-of-Order Message Tolerance

Diaspora users frequently experience message reordering due to multi-hop relay paths and intermittent connectivity. We extend the message key cache window from Signal's default of 2,000 to a configurable maximum of 10,000, with a time-based eviction policy (default: 7 days) to bound memory usage.

3.2.3 Key Backup and Recovery

Device loss is a critical concern for users in transit or under coercion. We provide an optional key backup mechanism using Shamir's Secret Sharing [Shamir, 1979]:

Algorithm 2 Distributed Key Backup

Require: Identity key ik, threshold t, share count n, trusted contacts $C = \{c_1, \ldots, c_n\}$
Ensure: Key shares distributed to contacts

```
entropy ← Serialize(ik)
shares ← SSS.Split(entropy, t, n)
for i ∈ {1, ..., n} do
    enc_share_i ← Encrypt(shares[i], DH(ik, c_i.pk))
    Send(c_i, enc_share_i)
end for
```

3.3 Group Messaging Protocol

We adopt a hybrid approach combining Sender Keys for efficiency with periodic full ratchet refreshes for post-compromise security.

Definition 2 (Group Epoch)**.** *A group epoch e is a period during which a fixed set of sender keys is valid. Epoch transitions occur on member addition, member removal, or after a configurable time interval T_e (default: 24 hours).*

3.3.1 Epoch Initialization

When a new epoch begins, the epoch initiator (typically the member who triggered the transition) generates a new group key:

Algorithm 3 Group Epoch Initialization

Require: Group members $M = \{m_1, \ldots, m_n\}$, initiator m_i
Ensure: New epoch established with fresh group key

```
epoch_seed ← Random(32)
group_key ← HKDF(epoch_seed, "group-key", 32)
for m_j ∈ M \ {m_i} do
    enc_seed_j ← PairwiseEncrypt(m_i, m_j, epoch_seed)
    Send(m_j, ("epoch", enc_seed_j))
end for
sender_key_i ← HKDF(group_key, m_i.id, 32)
return (group_key, sender_key_i)
```

3.3.2 Group Message Sending

Within an epoch, messages are encrypted using the sender's derived key with a symmetric ratchet for forward secrecy:

Algorithm 4 Group Message Send

Require: Sender m_i, plaintext p, current sender key state $(sk_i, counter)$
Ensure: Ciphertext c and updated state

```
(mk, sk'_i) ← KDF(sk_i, "msg-key")
nonce ← Counter2Nonce(counter)
c ← AEAD.Enc(mk, nonce, p, m_i.id || counter)
counter ← counter + 1
sk_i ← sk'_i
return (c, m_i.id, counter − 1)
```

3.4 Distributed Store-and-Forward Network

The DSF network provides message persistence without centralized servers. It consists of volunteer relay nodes organized into a Kademlia-based DHT [Maymounkov and Mazières, 2002] with the following modifications:

3.4.1 Mailbox Allocation

Each user is assigned a set of k relay nodes (default $k = 5$) as their *mailbox set*. Mailbox assignment uses consistent hashing on the user's long-term public key:

$$mailbox(pk) = \{n \in \mathcal{N} : \mathsf{dist}(\mathcal{H}(pk), n.id) \leq \mathsf{threshold}(k, |\mathcal{N}|)\} \tag{4}$$

where $\mathcal{N}$ is the set of active relay nodes and dist is the XOR distance metric.

3.4.2 Message Storage and Retrieval

Algorithm 5 DSF Message Deposit

Require: Ciphertext c, recipient identifier rid
Ensure: Message stored at recipient's mailbox nodes

```
nodes ← FindMailbox(rid)
msg_id ← H(c || rid || timestamp())
ttl ← 7 × 24 × 3600                          ▷ 7-day default TTL
envelope ← (msg_id, c, rid, timestamp(), ttl)
ack_count ← 0
for n ∈ nodes do
    if Store(n, envelope) then
        ack_count ← ack_count + 1
    end if
end for
if ack_count < ⌈k/2⌉ then
    retry with expanded node set
end if
```

3.4.3 Offline Retrieval

When a user comes online after an offline period, they query their mailbox nodes:

Algorithm 6 DSF Message Retrieval

Require: User key pair (sk, pk), last sync timestamp t_{last}
Ensure: All pending messages retrieved and decrypted

```
nodes ← FindMailbox(H(pk))
msgs ← ∅
for n ∈ nodes do
    batch ← Fetch(n, H(pk), t_last)
    msgs ← msgs ∪ batch
end for
msgs ← Deduplicate(msgs)                ▷ By msg_id
msgs ← SortByTimestamp(msgs)
for m ∈ msgs do
    p ← Decrypt(sk, m.ciphertext)
    Deliver(p)
end for
AcknowledgeRetrieval(nodes, msgs)
```

3.5 Anti-Spam System

ParsMsg employs a layered anti-spam system that operates without personal identity verification.

3.5.1 Layer 1: Proof-of-Work

Every first-contact message must include a proof-of-work token:

Definition 3 (Contact PoW)**.** *A valid proof-of-work for sender s contacting recipient r is a nonce w such that:*

$$\mathcal{H}(s.pk \| r.pk \| w \| \lfloor t/T_w \rfloor) < D \tag{5}$$

where D is the difficulty target and T_w is the time window (default: 1 hour). The difficulty D is calibrated to require approximately 5 seconds of computation on a modern mobile device.

3.5.2 Layer 2: Social Trust Amplification

Users who are vouched for by existing contacts receive reduced PoW requirements:

$$D_{effective}(s, r) = D \cdot 2^{\min(trust(s,r),10)} \tag{6}$$

where $trust(s, r)$ is the shortest path length in the social trust graph between s and r, and the exponent is capped at 10 to prevent complete PoW elimination.

3.5.3 Layer 3: Rate Limiting

Each relay node enforces per-sender rate limits based on accumulated PoW credits:

Algorithm 7 Rate Limit Check

Require: Sender s, message m, relay node n
Ensure: Accept or reject message

```
credits ← n.GetCredits(s.pk)
cost ← MessageCost(|m|)                    ▷ Proportional to size
if credits ≥ cost then
    n.DeductCredits(s.pk, cost)
    return ACCEPT
else
    return REJECT
end if
```

3.6 Media Sharing

Large media files (images, voice messages, videos) are handled through a content-addressed storage layer integrated with the DSF network:

1. The sender encrypts the media file with a random symmetric key k_m.
2. The encrypted media is split into fixed-size chunks (256 KB) and distributed across multiple relay nodes using erasure coding (Reed-Solomon with $n = 2k$ redundancy).
3. The message body contains the decryption key k_m, the content hash, and the chunk manifest.
4. The recipient retrieves chunks from available nodes and reconstructs the media.

This design ensures that no single relay node possesses the complete encrypted media, and the decryption key is protected by the end-to-end encryption of the message itself.

4 Security Analysis

4.1 Forward Secrecy

Theorem 1 (Forward Secrecy)**.** *If the DDH assumption holds in $\mathbb{G}$, then ParsMsg provides forward secrecy: compromise of a user's long-term identity key does not reveal the contents of previously exchanged messages.*

Proof. Forward secrecy follows from the DH ratchet construction. Each ratchet step generates an ephemeral DH key pair $(sk_e, pk_e) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and derives the chain key from $\mathsf{DH}(sk_e, pk_r)$ where pk_r is the peer's current ratchet public key. After the ratchet step, sk_e is erased. An adversary who later obtains the long-term key cannot compute $\mathsf{DH}(sk_e, pk_r)$ without sk_e, which is equivalent to solving the CDH problem in $\mathbb{G}$. Under DDH, CDH is hard, completing the proof. □

4.2 Post-Compromise Security

Theorem 2 (Post-Compromise Security)**.** *If the DDH assumption holds, ParsMsg provides post-compromise security: after a device compromise, security is restored within at most two DH ratchet steps following the compromise.*

Proof. Suppose the adversary compromises the device state at time t, obtaining the current ratchet state (s_t, dh_t). After the next DH ratchet step, a fresh key pair dh_{t+1} is generated and the new root key s_{t+1} incorporates $\mathsf{DH}(dh_{t+1}.sk, pk_{peer})$. Since $dh_{t+1}.sk$ is unknown to the adversary

(generated after compromise), computing s_{t+1} requires solving CDH. After the peer also performs a ratchet step (generating dh_{t+2} on their side), both directions are secured with keys unknown to the adversary. □

4.3 Group Security Properties

Property 1 (Group Forward Secrecy)**.** *Within an epoch, the symmetric sender key ratchet provides forward secrecy for individual messages. Across epochs, the full re-keying procedure provides forward secrecy against an adversary who compromises a future epoch's group key.*

Property 2 (Group Post-Compromise Security)**.** *Epoch transitions triggered by the time-based policy (T_e) ensure that group security recovers from member compromise within at most T_e time, assuming the compromised member does not collude with the adversary beyond the compromise point.*

4.4 Metadata Protection

Theorem 3 (Relay Node Metadata Limitation)**.** *A relay node that is not a mailbox node for both sender and recipient learns at most the recipient's mailbox identifier (a hash of their public key) and the encrypted message size. It cannot determine the sender's identity.*

Proof. Messages deposited in the DSF network are addressed by $\mathcal{H}(pk_r)$, which reveals no information about the sender. The message body, including sender identification, is encrypted under the end-to-end encryption layer. The relay node sees only the envelope: $(msg_id, c, rid, timestamp, ttl)$, where c is the ciphertext and $rid = \mathcal{H}(pk_r)$. □

5 Persian Web Traffic Camouflage

A key contribution of ParsMsg is a pluggable transport designed to blend messaging traffic with legitimate Persian web browsing patterns.

5.1 Traffic Profile Analysis

We analyzed traffic patterns from 500 consenting users over 30 days to characterize legitimate Persian web traffic. Key findings:

Table 1: Persian Web Traffic Characteristics

Characteristic	**Mean**	**Std Dev**
Packet size (bytes)	847	312
Inter-packet delay (ms)	234	187
Session duration (min)	12.3	8.7
Requests per session	47	31
TLS record size (bytes)	1,203	456

5.2 Camouflage Transport Design

The camouflage transport operates by:

1. Establishing a TLS 1.3 connection with an ESNI (Encrypted Server Name Indication) extension to a cooperating front server hosting legitimate Persian content.
2. Encoding messaging data into HTTP/2 request/response streams that statistically match the traffic profile in Table 1.

3. Injecting cover traffic during idle periods to maintain consistent traffic patterns.
4. Using steganographic encoding in image requests to popular Persian image hosting services as a fallback channel.

6 Implementation

6.1 Cryptographic Primitives

Table 2: Cryptographic Primitive Selection

Function	**Primitive**	**Rationale**
Key Agreement	X25519 [Bernstein, 2006]	Performance, security
Signatures	Ed25519 [Bernstein et al., 2012]	Compact, fast
AEAD	XChaCha20-Poly1305	Nonce-misuse resistance
Hash	BLAKE3	Speed, parallelism
KDF	HKDF-BLAKE3	Consistency with hash
SSS	GF(2^8) Shamir	Standard, well-analyzed
Erasure Coding	Reed-Solomon (16,32)	Mature implementations

6.2 Client Implementation

The ParsMsg client is implemented as a cross-platform library in Rust with bindings for iOS (Swift), Android (Kotlin), and desktop (Electron/TypeScript). Key implementation details:

- **State Management:** All cryptographic state is stored in an encrypted SQLite database using SQLCipher, with the database key derived from the user's device passcode via Argon2id [Biryukov et al., 2016].
- **Memory Safety:** Sensitive key material is allocated on locked, non-swappable memory pages and securely erased (zeroized) when no longer needed.
- **Binary Size:** The core library compiles to 2.8 MB (ARM64), suitable for distribution in bandwidth-constrained environments.

6.3 Relay Node Implementation

Relay nodes are implemented in Go for deployment simplicity:

- **Storage:** Messages are stored in an embedded BadgerDB [BadgerDB, 2023] key-value store with automatic TTL-based expiration.
- **Networking:** Nodes communicate using QUIC [Langley et al., 2017] for low-latency multiplexed connections with built-in encryption.
- **DHT:** The Kademlia DHT is implemented using libp2p [Dias et al., 2022] with custom extensions for mailbox routing.
- **Resource Requirements:** A relay node requires approximately 512 MB RAM and 10 GB disk space to serve 10,000 mailboxes.

7 Evaluation

7.1 Testnet Deployment

We deployed a testnet of 2,400 nodes across 14 countries: United States (480), Canada (360), Germany (240), United Kingdom (240), Sweden (120), Australia (120), Turkey (120), UAE (120), Iran (via cooperating VPN endpoints, 120), France (120), Netherlands (120), Austria (60), Norway (60), and Japan (60).

7.2 Message Delivery Latency

Table 3: Message Delivery Latency (milliseconds)

Scenario	**P50**	**P90**	**P95**	**P99**	**Max**
Both online, same region	87	142	178	312	890
Both online, cross-region	340	520	680	1,240	3,100
Recipient offline 1h	3,612	5,230	6,100	8,900	12,400
Recipient offline 24h	86,450	86,890	87,200	88,100	92,000
Recipient offline 72h	259,300	259,800	260,100	261,000	268,000
Via camouflage transport	1,240	2,100	2,890	4,500	8,200

7.3 Message Delivery Reliability

Over a 30-day evaluation period with simulated offline periods:

Table 4: Message Delivery Reliability

Offline Duration	**Delivery Rate**	**Duplicate Rate**
0 (always online)	100.0%	0.0%
$\leq$ 1 hour	100.0%	0.1%
$\leq$ 24 hours	99.97%	0.3%
$\leq$ 72 hours	99.7%	0.8%
$\leq$ 7 days	98.2%	1.4%

7.4 Anti-Spam Effectiveness

We simulated spam attacks with varying strategies:

Table 5: Anti-Spam System Performance

Attack Strategy	**Messages/hr**	**Blocked**	**False Pos.**
Brute force (no PoW)	10,000	100.0%	0.0%
PoW-equipped, no trust	200	94.3%	0.0%
PoW + fake trust graph	500	99.2%	0.1%
Sybil (1,000 identities)	5,000	98.8%	0.2%
Legitimate first contact	50	0.0%	0.0%
Legitimate established	500	0.0%	0.0%

7.5 Group Messaging Performance

Table 6: Group Epoch Transition Overhead

Group Size	Rekey Time (ms)	Bandwidth (KB)	Messages/epoch
5	12	2.4	847
20	45	9.6	3,420
50	110	24.0	8,100
100	230	48.0	15,600
500	1,180	240.0	72,000
1,000	2,400	480.0	134,000

7.6 Comparison with Existing Systems

Table 7: Feature Comparison with Existing Messaging Systems

Feature	ParsMsg	Signal	Matrix	Briar	Session
E2E Encryption	Yes	Yes	Yes (Olm)	Yes	Yes
Forward Secrecy	Yes	Yes	Yes	Yes	No
Post-Compromise	Yes	Yes	No	Yes	No
Decentralized	Yes	No	Partial	Yes	Yes
Offline Delivery	Yes (7d)	No	Yes	No	Yes (14d)
No Phone/Email	Yes	No	Yes	Yes	Yes
Group PCS	Yes	No	No	No	No
Traffic Camouflage	Yes	No	No	Tor	No
Anti-Spam (no ID)	Yes	No	Partial	Yes	Partial

8 Deployment Considerations

8.1 Relay Node Incentives

Relay node operators are compensated through the Pars network's ASHA token system. Each successfully delivered message generates a micro-payment to the relay nodes involved in its delivery chain. The payment is proportional to storage duration and bandwidth consumed, with a current rate of approximately 0.001 ASHA per message-day of storage.

8.2 Client Distribution

In jurisdictions where app store distribution is restricted, ParsMsg clients are distributed through:

1. Direct APK/IPA distribution via the Pars network's decentralized content delivery system.
2. Domain-fronted download endpoints that appear as legitimate HTTPS traffic to popular CDN providers.
3. Physical distribution via Bluetooth and local Wi-Fi sharing between trusted contacts.
4. QR code-based progressive web app (PWA) installation for minimal-footprint deployment.

8.3 Regulatory Compliance

ParsMsg is designed to comply with lawful intercept requirements in democratic jurisdictions through transparent architecture: since the system is end-to-end encrypted with no backdoors, compliance consists of documenting that lawful access requires cooperation of the target user's device, consistent with established legal frameworks for encrypted communications [Abelson et al., 2015].

9 Limitations and Future Work

1. **Global Traffic Analysis:** ParsMsg does not protect against an adversary capable of simultaneously observing all network links. Mixing networks could address this at the cost of increased latency.

2. **Large Groups:** Epoch transitions for groups exceeding 1,000 members incur significant overhead. TreeKEM-based approaches from MLS could reduce this to $O(\log n)$.

3. **Voice/Video:** Real-time communication requires extensions beyond the current store-and-forward model. We are investigating WebRTC integration with the camouflage transport.

4. **Key Transparency:** The current design relies on trust-on-first-use (TOFU) for key verification. A key transparency log [Melara et al., 2015] would strengthen this.

5. **Quantum Resistance:** The X25519 key agreement is not quantum-resistant. Hybrid key encapsulation with ML-KEM (Kyber) [Bos et al., 2018] is planned for a future protocol version.

10 Conclusion

We have presented ParsMsg, a messaging protocol designed for the specific needs of the Persian diaspora community operating under adversarial network conditions. By combining an extended Double Ratchet protocol with a distributed store-and-forward network, traffic camouflage, and identity-free anti-spam mechanisms, ParsMsg provides strong security guarantees while maintaining practical usability. Our testnet evaluation demonstrates that the protocol achieves acceptable performance for real-world deployment, with sub-second delivery for online users and reliable delivery for users experiencing extended offline periods.

The ParsMsg protocol is open source under the Apache 2.0 license, and we invite the broader security and privacy community to audit and contribute to its development.

References

Anderson, C. and Murdoch, S. J. The dimming of the internet: Detecting throttling as a mechanism of censorship in Iran. *arXiv preprint arXiv:1306.4361*, 2013.

Angel, Y., Kadianakis, G., and Murdoch, S. J. obfs4 specification. *Tor Project Technical Report*, 2014.

Abelson, H., Anderson, R., Bellovin, S. M., et al. Keys under doormats: Mandating insecurity by requiring government access to all data and communications. *Journal of Cybersecurity*, 1(1):69–79, 2015.

Back, A. Hashcash – a denial of service counter-measure. Technical report, 2002.

Dgraph Labs. BadgerDB: Fast key-value DB in Go. `https://github.com/dgraph-io/badger`, 2023.

Barnes, R., Beurdouche, B., Robert, R., et al. The Messaging Layer Security (MLS) Protocol. *RFC 9420*, 2023.

Bernstein, D. J. Curve25519: New Diffie-Hellman speed records. In *PKC 2006*, pages 207–228, 2006.

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., and Yang, B.-Y. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

Biryukov, A., Dinu, D., and Khovratovich, D. Argon2: New generation of memory-hard functions for password hashing and other applications. In *IEEE EuroS&P*, pages 292–302, 2016.

Bos, J., Ducas, L., Kiltz, E., et al. CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM. In *IEEE EuroS&P*, pages 353–367, 2018.

Bozorgmehr, M. Iranian immigration and the changing landscape of American diversity. In *Iranians in Texas*, Univ. of Texas Press, 2007.

Briar Project. Briar: Secure messaging, anywhere. `https://briarproject.org`, 2018.

Chase, M. and Perrin, T. The Signal Private Group System. Technical Report, Signal Foundation, 2020.

Deibert, R. J., Palfrey, J. G., Rohozinski, R., and Zittrain, J. *Access Denied: The Practice and Policy of Global Internet Filtering*. MIT Press, 2008.

Dias, D., Benet, J., and Protocol Labs. libp2p: A modular network stack. `https://libp2p.io`, 2022.

Dingledine, R., Mathewson, N., and Syverson, P. Tor: The second-generation onion router. In *USENIX Security Symposium*, 2004.

Electronic Frontier Foundation. Digital privacy at the U.S. border. `https://www.eff.org/wp/digital-privacy-us-border`, 2017.

Ermoshina, K., Musiani, F., and Halpin, H. End-to-end encrypted messaging protocols: An overview. In *INSCI*, pages 244–254, 2017.

Fifield, D., Lan, C., Hynes, R., Wegmann, P., and Paxson, V. Blocking-resistant communication through domain fronting. In *PETS*, pages 46–64, 2015.

Fifield, D. meek pluggable transport. Technical report, Tor Project, 2015.

Jacob, F. and Eikenberg, R. Analysis of the Matrix event graph replicated data type. *arXiv preprint arXiv:1907.12539*, 2019.

Karlin, J., Ellard, D., Jackson, A. W., et al. Decoy routing: Toward unblockable internet communication. In *USENIX FOCI*, 2011.

Langley, A., Riddoch, A., Wilk, A., et al. The QUIC transport protocol: Design and internet-scale deployment. In *ACM SIGCOMM*, pages 183–196, 2017.

Marczak, B., Guarnieri, C., Marquis-Boire, M., and Scott-Railton, J. Hacking Team and the targeting of Ethiopian journalists. *Citizen Lab Research Report*, 2014.

Marlinspike, M. and Perrin, T. The Signal Protocol. Technical report, Signal Foundation, 2016.

Maymounkov, P. and Mazières, D. Kademlia: A peer-to-peer information system based on the XOR metric. In *IPTPS*, pages 53–65, 2002.

Melara, M. S., Blankstein, A., Bonneau, J., Felten, E. W., and Freedman, M. J. CONIKS: Bringing key transparency to end users. In *USENIX Security*, pages 383–398, 2015.

Mislove, A., Post, A., Gummadi, K. P., and Druschel, P. Ostra: Leveraging trust to thwart unwanted communication. In *NSDI*, pages 15–30, 2008.

Perrin, T. and Marlinspike, M. The Double Ratchet Algorithm. Technical Report, Signal Foundation, 2016.

Shamir, A. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

Winter, P. and Lindskog, S. How the Great Firewall of China is blocking Tor. In *USENIX FOCI*, 2012.

Wustrow, E., Wolchok, S., Goldberg, I., and Halderman, J. A. Telex: Anticensorship in the network infrastructure. In *USENIX Security*, pages 459–474, 2011.

A Protocol Message Formats

A.1 Envelope Format

Listing 1: DSF Envelope Binary Format

```
Envelope {
    version:    uint8       // Protocol version (1)
    msg_id:     [32]byte    // BLAKE3 hash
    recipient:  [32]byte    // H(pk_recipient)
    timestamp:  uint64      // Unix timestamp (ms)
    ttl:        uint32      // Time-to-live (seconds)
    flags:      uint16      // Delivery flags
    payload:    []byte      // Encrypted message
    pow_nonce:  [8]byte     // Anti-spam PoW
    pow_hash:   [32]byte    // PoW verification
}
```

A.2 Message Format

Listing 2: Encrypted Message Format

```
Message {
    header:     []byte      // Encrypted DR header
    ciphertext: []byte      // AEAD encrypted body
    media_refs: []MediaRef  // Optional media references
}

MediaRef {
    content_hash: [32]byte  // BLAKE3 of plaintext
    enc_key:      [32]byte  // Symmetric media key
    chunk_count:  uint16    // Number of chunks
    total_size:   uint64    // Original file size
    mime_type:    string    // MIME type
    manifest:     []ChunkLoc // Chunk locations
}
```

B Trust Graph Construction

The social trust graph is constructed through mutual key verification ceremonies. When two users verify each other's keys (via QR code scanning, safety number comparison, or in-person verification), a bidirectional trust edge is created. Trust edges are stored locally and propagated through the network using a gossip protocol with the following properties:

- Trust edges are signed by both parties using their identity keys.
- Trust propagation is limited to 3 hops to bound graph exploration.
- Trust edge creation is rate-limited to 10 new edges per day per user.
- Revocation of trust edges takes effect within 1 hour across the network.